Task 5(A): Reinforcement Learning

Prepared By Aamna Ansari

1. Introduction

Reinforcement Learning (RL) is a way of teaching a computer program, called an agent, to make good decisions by trial and error. The agent takes actions in an environment, receives rewards or penalties, and slowly learns which actions give the best results. RL is used in games, robotics, recommendation systems, and in the training of modern AI assistants.

This report covers the following parts of Task 5(A):

- The core concepts of RL, including the agent-environment interaction and the Markov Decision Process (MDP).
- Six major RL algorithms: Q-Learning, SARSA, DQN, Policy Gradient methods, Actor-Critic methods and PPO. Each is explained in the same format.
- A practical implementation of Q-Learning from scratch on the Taxi-v3 environment from Gymnasium, with training, testing and results.
- A short discussion of how RL connects to AI Agents and Agentic AI.

2. Core Concepts of Reinforcement Learning

2.1 What is Reinforcement Learning?

Reinforcement Learning (RL) is a type of machine learning where an agent learns to make decisions by trying actions and seeing the results. The agent is not told the correct answer. Instead, it gets feedback in the form of rewards and penalties, and it slowly learns which actions lead to the best results over time [1].

A simple example is training a dog. If the dog sits when you say “sit”, it gets a treat (reward). If it does something wrong, it gets no treat. After many tries, the dog learns what to do. RL works in the same way, but the learner is a computer program.

RL is different from other types of learning:

Type of learning	What the learner gets
Supervised learning	Examples with correct answers
Unsupervised learning	Data without answers (finds patterns)
Reinforcement learning	Rewards or penalties after taking actions

The main goal of an RL agent is to find a strategy that gives the highest total reward in the long run.

2.2 How Reinforcement Learning Works

RL works as a loop between an agent and its environment, as shown in Figure 1. At each time step, the agent looks at the current state, chooses an action, and the environment replies with a new state and a reward. The agent uses this feedback to improve. This repeats many times.

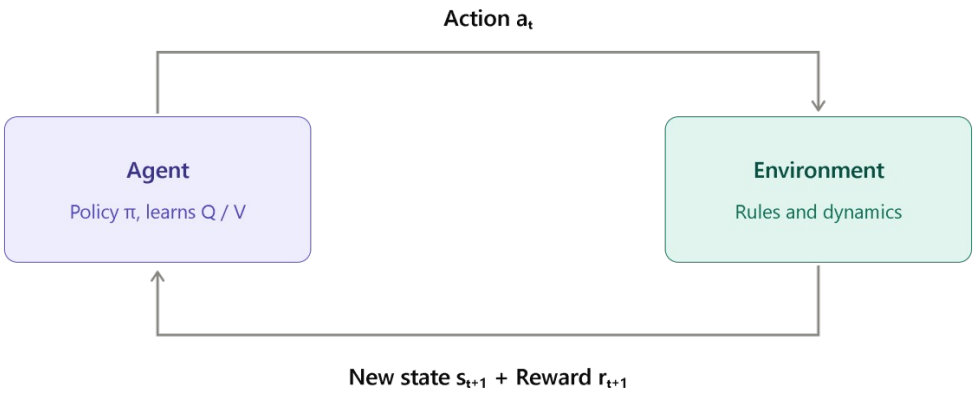


Figure 1: Agent-Environment Interaction Loop

Explanation of the Interaction

Reinforcement Learning is based on a continuous feedback loop between an agent and its environment. At each time step *t*, the following sequence takes place:

1. The agent observes the current state s_t of the environment.
2. Based on its policy π, the agent selects an action a_t.
3. The environment responds to this action and moves to a new state s_{t+1}.
4. The environment also returns a numerical reward r_{t+1}, which tells the agent how good or bad the action was.
5. The agent uses this feedback to update its policy or value estimates, and the loop repeats.

The agent's objective is not to maximise the immediate reward, but the cumulative discounted return over time:

$G_t = r_{t+1} + \gamma \cdot r_{t+2} + \gamma^2 \cdot r_{t+3} + \ldots$, where γ ($0 \leq \gamma \leq 1$) is the discount factor.

Role of Each Component

Component	Role in the loop
Agent	The learner and decision-maker. It chooses actions according to its policy.
Environment	Everything outside the agent. It reacts to actions and produces the next state and reward.
State (s)	A description of the current situation of the environment.
Action (a)	A choice made by the agent from the set of available actions.
Reward (r)	A scalar feedback signal. Positive values act as rewards, negative values as penalties.

An important point is that the agent does not only care about the reward of one step. A small reward now may lead to a big reward later, and a big reward now may lead to trouble later. So the agent tries to maximise the total reward over time, which is called the return.

2.3 Agent and Environment

Agent. The agent is the learner and decision-maker. It observes the situation, chooses actions, and updates its knowledge from the feedback it receives. In our project, the agent is the taxi driver (the Q-Learning program).

Environment. The environment is everything outside the agent that the agent interacts with. It contains the rules of the world. It reacts to the agent's actions and gives back the next state and the reward. In our project, the environment is the Taxi-v3 grid world.

The agent cannot change the rules of the environment. It can only choose actions and learn from what happens.

Real-World Example 1: Robot Vacuum Cleaner

Element	In the robot vacuum
Agent	The control software of the robot
Environment	The room, including furniture, walls, dirt and the charging dock
State	Robot position, sensor readings (obstacle nearby or not), battery level, which areas are already cleaned
Action	Move forward, turn left, turn right, start suction, return to dock
Reward	+1 for cleaning a dirty tile, -1 for hitting an obstacle, -0.1 per step (to encourage efficiency), +10 for docking before the battery dies

At each step, the robot senses its surroundings (state), chooses a movement (action), and receives feedback (reward) based on what happened. Over many cleaning runs (episodes), it learns a policy that covers the room quickly without collisions and returns to the dock in time.

Real-World Example 2: Taxi-v3 (the environment used in the implementation)

Element	In Taxi-v3
Agent	The taxi driver
Environment	A 5×5 grid world with four pickup/drop-off locations
State	Taxi position, passenger location, destination (500 discrete states)
Action	South, North, East, West, Pickup, Drop-off (6 actions)
Reward	+20 for a successful drop-off, -1 per step, -10 for an illegal pickup or drop-off

2.4 States

A state is a description of the current situation of the environment. It should contain the information the agent needs to decide what to do next.

In Taxi-v3, a state tells us three things: where the taxi is, where the passenger is, and where the passenger wants to go. There are 500 possible states, and each one is stored as a number from 0 to 499. For example, the state number 386 in our test run means: taxi at row 3, column 4, passenger at Green, destination Yellow.

The set of all possible states is called the state space.

2.5 Actions

An action is a choice that the agent can make in a state. The set of all possible actions is called the action space.

In Taxi-v3, the agent has 6 actions: move South, North, East or West, pick up the passenger, and drop off the passenger. The action space can be:

- **Discrete:** a limited list of choices, like in Taxi-v3.
- **Continuous:** values in a range, like the angle of a robot arm.

2.6 Rewards and Penalties

A reward is a number that the environment gives to the agent after each action. It tells the agent how good or bad that action was.

- A positive reward encourages the agent to repeat the behaviour.
- A negative reward (also called a penalty) discourages the behaviour.

The agent does not know in advance which actions are good. It finds out only by receiving rewards. In Taxi-v3, the rewards are:

Event	Reward
Every normal step	−1
Correct drop-off	+20
Illegal pickup or drop-off	−10

The reward design is very important. The −1 per step makes the agent finish the job quickly, and the −10 teaches it to avoid wrong pickups and drop-offs.

Reward and return. The reward is the feedback for one step. The return is the total of all future rewards, with later rewards counted a little less. It is written as:

$$G_t = r_{t+1} + \gamma \cdot r_{t+2} + \gamma^2 \cdot r_{t+3} + \dots$$

Here γ (gamma) is the discount factor, a number between 0 and 1. If γ is close to 1, the agent cares a lot about the future. If γ is close to 0, the agent cares mostly about the immediate reward. Our experiment showed this clearly: with γ = 0.5, the agent performed worse than with γ = 0.99.

2.7 Policy

A policy is the agent's strategy. It tells the agent which action to take in each state. It is written as π (pi).

- A **deterministic policy** always gives one action for a state: π(s) = a.
- A **stochastic policy** gives a probability for each action: π(a | s).

The goal of RL is to find the optimal policy, which is the policy that gives the highest expected return.

In our project, the policy at the end of training is simple: in every state, choose the action with the highest value in the Q-table. In other words, π(s) = argmax Q(s, a).

2.8 Value Function

The value function V(s) tells how good it is to be in a state. It is the expected return that the agent can get starting from that state and following its policy.

$$V(s) = \text{expected value of } G_t, \text{ starting from state } s$$

A state with a high value is a good place to be. In Taxi-v3, a state where the taxi is already at the passenger with the correct destination in mind has a higher value than a state where the taxi is far away, because the reward is closer and fewer −1 penalties will be collected.

The value function helps the agent judge situations, not just actions.

2.9 Q-Value (Action-Value Function)

The Q-value Q(s, a) tells how good it is to take a specific action a in a specific state s, and then continue following the policy. It is the expected return after taking that action.

The difference from V(s) is simple:

	What it measures
V(s)	How good is this state?
Q(s, a)	How good is this action in this state?

The two are connected. For the best policy, V(s) is the Q-value of the best action in that state:

$$V^*(s) = \max Q^*(s, a)$$

Q-values are useful because the agent can choose an action directly: it looks at all Q(s, a) values for its current state and picks the highest one. This is what our Q-table stores. It has 500 rows (states) and 6 columns (actions), so it holds 500 × 6 = 3,000 Q-values.

2.10 Exploration vs Exploitation

The agent faces a basic problem:

- **Exploitation:** choose the action that looks best based on what it already knows.
- **Exploration:** try a new or less-known action, because it might be even better.

If the agent only exploits, it may get stuck with a poor strategy because it never discovers better options. If it only explores, it never uses what it has learned. A good agent needs a balance.

A common method is the ε-greedy strategy. With probability ε, the agent takes a random action (explore). With probability 1 − ε, it takes the best known action (exploit). Usually ε starts high and is reduced slowly (ε-decay), so the agent explores a lot at the start and exploits more later.

In our project, ε starts at 1.0 and is multiplied by 0.999 after every episode, until it reaches a minimum of 0.01.

2.11 Episodes and Steps

A **step** is one interaction: the agent takes one action, and the environment returns a new state and a reward.

An **episode** is one complete run from a starting state until the task ends. In Taxi-v3, an episode ends in one of two ways:

- **Terminated:** the passenger is dropped at the correct destination.
- **Truncated:** the time limit of 200 steps is reached.

The agent learns over many episodes. In our project, we trained for 10,000 episodes. After training, the agent finished an episode in about 13 steps on average, while a random agent used about 194 steps.

2.12 Markov Decision Process (MDP)

What is an MDP?

A Markov Decision Process (MDP) is the mathematical framework used to describe an RL problem [1]. Almost all RL problems, including Taxi-v3, can be written as an MDP. It is defined by five parts:

(S, A, P, R, γ)

Symbol	Name	Meaning
S	States	The set of all possible states
A	Actions	The set of all possible actions
P	Transition probability	P(s' | s, a): the chance of moving to state s' after taking action a in state s
R	Reward function	The reward received after taking action a in state s
γ	Discount factor	How much future rewards are valued (0 to 1)

The Markov Property

The word "Markov" refers to an important idea: the next state depends only on the current state and the action taken, not on the older history. In simple words, the current state already contains all the information needed to decide what to do. How the agent reached this state does not matter.

For example, in Taxi-v3, to decide the next move, the agent only needs to know the taxi position, the passenger location and the destination. It does not need to remember the path it took to get there.

The Goal in an MDP

The goal is to find the best policy, one that maximises the expected return. The best Q-values follow the Bellman optimality equation [1]:

$$Q^*(s, a) = \Sigma\, P(s' \mid s, a) \cdot [\, R(s, a, s') + \gamma \cdot \max Q^*(s', a') \,]$$

In words: the value of an action equals the immediate reward plus the discounted value of the best action in the next state. Q-Learning does not solve this equation directly. Instead, it learns the values step by step from experience, using the update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\, [\, r + \gamma \cdot \max Q(s',a') - Q(s,a) \,]$$

You can see that the update moves the old Q-value towards "reward + discounted best future value", which is the right-hand side of the Bellman equation.

Taxi-v3 as an MDP

MDP part	In Taxi-v3
States (S)	500 states: taxi position (25) × passenger location (5) × destination (4)
Actions (A)	6 actions: South, North, East, West, Pickup, Drop-off
Transition (P)	Deterministic: the same action in the same state always gives the same next state, so P(s' | s, a) is 1 for one particular s' and 0 for all others
Reward (R)	−1 per step, +20 for a correct drop-off, −10 for an illegal pickup or drop-off
Discount factor (γ)	0.99 (our choice for training)
Start	The taxi, passenger and destination are chosen randomly at the beginning of each episode
End	Terminated after a correct drop-off, or truncated after 200 steps

Because Taxi-v3 is deterministic and has a small number of states, it is a good example for learning RL with a simple Q-table. In bigger problems, the transitions can be random (for example, a robot's wheel may slip), and the states can be too many for a table. This is why methods like DQN and PPO are needed.

3. Algorithms

3.1 Q-Learning

Point	Details
1. How it works	Q-Learning is a value-based RL algorithm [2]. The agent keeps a table called the Q-table. It stores a value Q(s, a) for every state-action pair, which tells how good it is to take action *a* in state *s*. The agent plays, gets rewards, and keeps updating the table. After enough training, the agent simply picks the action with the highest Q-value in each state.
2. Architecture / learning mechanism	The core is a table of size (number of states × number of actions). For example, Taxi-v3 has a 500 × 6 table. The update rule is: `Q(s,a) ← Q(s,a) + α [ r + γ · max Q(s',a') − Q(s,a) ]` Here α is the learning rate, γ is the discount factor, r is the reward and s' is the next state. The term `max Q(s',a')` uses the best possible next action, even if the agent did not actually take it. This makes Q-Learning an off-policy method. Actions during training are chosen with an ε-greedy strategy to balance exploration and exploitation.
3. Advantages	Simple and easy to understand and code. Needs no model of the environment (model-free). Off-policy, so it can learn the optimal policy while following an exploring policy. It is guaranteed to converge to the optimal Q-values in the tabular case, if every state-action pair is visited enough and the learning rate is reduced properly.
4. Limitations	Works only with small, discrete state and action spaces. The Q-table becomes too big for large problems (for example, images or continuous values). It cannot generalise: it learns each state separately. The max operator can cause overestimation of Q-values. It can learn slowly in large environments.
5. Suitable applications	Grid-world and maze problems, simple games (Taxi, FrozenLake, Tic-Tac-Toe), basic robot navigation, simple inventory or scheduling problems with few states.
6. Single-agent or	Designed mainly for single-agent problems. In multi-agent settings, if every agent

Point	Details
multi-agent	runs its own Q-Learning, the environment keeps changing for each agent because the others are also learning. This breaks the basic assumptions and can make learning unstable. Extensions such as Minimax-Q and Nash-Q were created for multi-agent games.
7. Use in AI Agents / Agentic AI	Useful for small decision problems inside an agent, such as choosing between a few tools, routing a request to one of a few handlers, or picking a response strategy, where the number of situations is small and feedback (success or failure) is clear. It is also a good teaching base for understanding how agents learn from rewards.

3.2 SARSA

Point	Details
1. How it works	SARSA is also a value-based, table-based algorithm, very similar to Q-Learning [3]. The name comes from the five items it uses in every update: **S**tate, **A**ction, **R**eward, next **S**tate, next **A**ction. The key idea is that the agent updates its Q-value using the action it actually takes next, not the best possible action. So it learns the value of the policy it is really following, including its exploratory moves.
2. Architecture / learning mechanism	It uses the same Q-table as Q-Learning. The update rule is: `Q(s,a) ← Q(s,a) + α [ r + γ · Q(s',a') − Q(s,a) ]` The difference from Q-Learning is that `max Q(s',a')` is replaced by `Q(s',a')`, where a' is the action chosen by the current policy (for example, ε-greedy). This makes SARSA an on-policy method. The agent must choose a' before it can do the update.
3. Advantages	Simple and easy to implement. Learns safer behaviour, because it takes its own exploration into account. Works well in environments where risky actions can give big penalties. Converges to the optimal policy if ε is slowly reduced towards zero.
4. Limitations	Same table limits as Q-Learning: only small, discrete problems, and no generalisation. Its result depends on the exploration strategy, so it is sensitive to the ε value. It is usually slower than Q-Learning in finding the truly optimal (but riskier) path. It cannot reuse old experience from a different policy (it is on-policy).
5. Suitable applications	Problems where safety matters: robot navigation near obstacles or hazards, the Cliff Walking problem, simple control tasks where mistakes are costly, and online learning where the agent learns while it works.
6. Single-agent or multi-agent	Mainly single-agent. It faces the same non-stationarity problem as Q-Learning when several agents learn together. Because it is on-policy, it can sometimes be a more stable choice in simple multi-agent experiments, but it is not designed for them.
7. Use in AI Agents / Agentic AI	Suitable for agents that must act carefully while still learning, such as a decision-making agent that handles sensitive actions (for example, deciding when to ask a human for approval instead of acting alone). Since it learns from the real behaviour of the agent, including its mistakes during exploration, it fits "learn while working" situations.

Key difference: Q-Learning vs SARSA

	Q-Learning	SARSA
Policy type	Off-policy	On-policy
Next-step value used	Best action: `max Q(s',a')`	Action actually taken: `Q(s',a')`
Behaviour	Bolder, learns the optimal path	More careful, learns a safer path
Cliff Walking example	Learns the shortest path near the cliff edge	Learns a longer, safer path away from the edge

3.3 Deep Q-Network (DQN)

Point	Details
1. How it works	DQN is Q-Learning combined with a deep neural network. When the number of states is huge (for example, game screens as images), a Q-table is impossible. So instead of a table, a neural network takes the state as input and predicts the Q-values for all actions. The agent then picks the action with the highest predicted value. DQN was introduced by DeepMind and became famous for learning to play Atari games directly from screen pixels [4].

2. Architecture / learning mechanism	**Q-network:** takes the state as input and gives one Q-value per action as output. For images, it uses convolutional layers (CNN); for simple vector states, it uses fully connected layers.
	Experience replay: the agent stores past experiences (s, a, r, s') in a memory buffer and trains on random batches from it. This breaks the link between consecutive samples and reuses data.
	Target network: a separate copy of the network, updated only every fixed number of steps. It gives stable training targets.
	Loss: the network is trained to reduce the difference between its prediction and the target:
	`target = r + γ · max Q_target(s',a')`
	`loss = ( target − Q(s,a) )²`
	(for a terminal state, target = r). Actions are chosen using ε-greedy.

3. Advantages	Can handle very large or continuous state spaces, including raw images. Generalises: similar states give similar Q-values. Experience replay makes learning more data-efficient and stable. It can reach human-level performance in many games. Many improved versions exist (Double DQN, Dueling DQN, Prioritized Replay, Rainbow).
4. Limitations	Works only with discrete actions (it needs a max over actions). Training can be unstable and sensitive to hyperparameters. Needs a lot of data and compute (sample inefficient). Can overestimate Q-values (fixed partly by Double DQN). Needs a replay buffer, which uses memory. Gives no convergence guarantee, unlike the tabular case.
5. Suitable applications	Video games (Atari), robot control with discrete actions, traffic signal control, recommendation systems, resource allocation, and other problems with complex states and a limited set of actions.
6. Single-agent or multi-agent	Originally for single-agent problems. For multi-agent problems, it is used in simple form (each agent has its own DQN, called independent learners), but the changing behaviour of other agents makes replay data outdated. Extensions like VDN and QMIX build on the DQN idea for cooperative multi-agent settings.
7. Use in AI Agents / Agentic AI	Fits agents that must choose from a fixed set of actions in a complex situation, such as a dialogue manager deciding the next conversation step, a scheduling agent

Point	Details
	assigning tasks, or an agent selecting tools based on a rich state description. It shows how deep learning can give agents the ability to learn from large, complex inputs. Note that modern LLM-based agents are usually trained with policy-based methods such as PPO, so DQN is more of a foundation than a common choice there.

Comparison table

Feature	Q-Learning	SARSA	DQN
Type	Value-based	Value-based	Deep value-based
Policy type	Off-policy	On-policy	Off-policy
Stores values in	Q-table	Q-table	Neural network
State space	Small, discrete	Small, discrete	Large, can be images
Action space	Discrete	Discrete	Discrete
Generalisation	None	None	Yes
Key technique	Max next Q	Actual next Q	Replay buffer + target network
Stability	Stable (tabular)	Stable (tabular)	Needs careful tuning
Best for	Learning basics, small problems	Safe learning	Complex problems, games
Multi-agent	Limited (Nash-Q, Minimax-Q)	Limited	Limited (QMIX, VDN)

3.4 Policy Gradient Methods

Point	Details
1. How it works	Policy Gradient methods are policy-based. Instead of learning Q-values and then picking the best action, the agent learns the policy directly. The policy is a function (usually a neural network) that takes a state and gives the probability of each action. The agent plays an episode, checks the total reward it got, and then changes the policy so that actions that led to high reward become more likely, and actions that led to low reward become less likely. The simplest and most well-known version is REINFORCE [5].
2. Architecture / learning mechanism	A policy network $\pi(a\|s; \theta)$ takes the state as input and outputs action probabilities (for discrete actions) or the mean and spread of a distribution (for continuous actions). The parameters θ are updated by gradient ascent: $\theta \leftarrow \theta + \alpha \cdot \nabla\theta \log \pi(a\|s; \theta) \cdot G_t$ Here G_t is the total discounted return from time step t. In simple words: if G_t is high, increase the probability of the action taken; if it is low, decrease it. A common improvement is to subtract a baseline (for example, the average return) from G_t. This reduces the variance of the updates without changing what the method learns on average. REINFORCE updates only after a full episode is finished (Monte Carlo style).
3. Advantages	Can handle continuous action spaces naturally. Can learn stochastic policies, which is useful when randomness is needed (for example, in games like rock-paper-scissors). Optimises the goal (expected return) directly. Works with large state

Point	Details
	spaces because it uses a neural network. Smooth changes in the policy usually give smooth changes in behaviour.
4. Limitations	High variance in the gradient estimates, so learning is noisy. Sample inefficient: it needs many episodes. It is on-policy, so old data cannot be reused after the policy changes. Waits until the end of an episode to learn, so it is slow for long episodes. Can get stuck in a local optimum. Sensitive to the learning rate: a bad step can make the policy much worse.
5. Suitable applications	Robot control with continuous actions (joint movements), simple games, problems that need a stochastic policy, and tasks where the action space is too large or continuous for value-based methods. It is mostly used as the foundation for stronger methods.
6. Single-agent or multi-agent	Mainly used for single-agent problems in its basic form. It can be applied to multi-agent problems, where each agent learns its own policy, but the high variance gets worse because other agents also change. Stronger multi-agent methods are usually built on its actor-critic versions.
7. Use in AI Agents / Agentic AI	The idea of directly improving a policy based on outcomes is the basis of how many modern agents are trained. For example, an agent that generates text or chooses actions can be improved by increasing the probability of outputs that got good feedback. In practice, newer and more stable methods built on this idea (such as PPO) are preferred over plain REINFORCE.

3.5 Actor-Critic Methods

Point	Details
1. How it works	Actor-Critic combines the value-based and policy-based approaches. It has two parts: the Actor, which decides which action to take (the policy), and the Critic, which judges how good the current state is (a value function). The critic gives feedback to the actor after every step, so the actor does not need to wait until the end of the episode. You can think of it as a student (actor) and a teacher (critic): the student tries actions, and the teacher says whether the result was better or worse than expected.
2. Architecture / learning mechanism	Two networks (or one network with two output heads): **Actor** $\pi(a\|s; \theta)$: outputs action probabilities. **Critic** $V(s; w)$: outputs the expected return from a state. After each step, the critic calculates the TD error: $\delta = r + \gamma \cdot V(s') - V(s)$ This tells whether things went better ($\delta > 0$) or worse ($\delta < 0$) than the critic expected. Then: Actor update: $\theta \leftarrow \theta + \alpha \cdot \nabla\theta \log \pi(a\|s; \theta) \cdot \delta$ Critic update: it is trained to reduce δ^2 (so its predictions become more accurate). The value δ is an estimate of the advantage, and it replaces the raw return G_t used in REINFORCE. This lowers the variance. Popular versions are A2C (Advantage Actor-Critic, synchronous) and A3C (Asynchronous Advantage Actor-Critic, which uses several parallel workers) [6].
3. Advantages	Lower variance than pure Policy Gradient, so learning is more stable and faster. Can learn after every step (online), not only after full episodes. Works with continuous and discrete actions. Keeps the benefits of policy-based methods (stochastic policies, large action spaces). Parallel versions (A3C, A2C) use multiple CPUs to speed up training.
4. Limitations	More complex: two networks must be trained together. The critic can be

Point	Details
	inaccurate, especially early in training, and then it gives bad guidance to the actor (this adds bias). Can be unstable if the actor and critic are not balanced well. Sensitive to hyperparameters. Basic versions are still on-policy and sample inefficient.
5. Suitable applications	Robotics and continuous control, games, autonomous driving simulations, resource management and trading simulations, and any task where stable learning with continuous actions is needed.
6. Single-agent or multi-agent	Works for single-agent problems, and it is also the most common base for multi-agent RL. A popular approach is centralized training with decentralized execution: during training, the critic can see information from all agents, while each actor uses only its own observations when acting. MADDPG and MAPPO follow this idea.
7. Use in AI Agents / Agentic AI	The actor-critic idea fits agentic systems well: one component decides the action, and another component evaluates the result. Similar roles appear in modern agent designs, such as a planner and an evaluator. In LLM training, a value model (critic) is often used along with the policy model (actor), for example in PPO-based fine-tuning.

3.6 Proximal Policy Optimization (PPO)

Point	Details
1. How it works	PPO is an actor-critic policy gradient method that makes training more stable [7]. A common problem in policy gradient methods is that one big update can ruin the policy. PPO solves this by limiting how much the policy is allowed to change in one update. The agent collects a batch of experience with the current policy, then improves the policy for a few epochs on that batch, but a “clipping” rule stops the new policy from moving too far from the old one.
2. Architecture / learning mechanism	PPO normally uses an actor network (policy) and a critic network (value function). Key parts: **Probability ratio:** `r(θ) = π_new(a|s) / π_old(a|s)` **Clipped objective:** `L = min( r(θ) · A, clip(r(θ), 1−ε, 1+ε) · A )` Here A is the advantage estimate and ε is a small number (commonly 0.2). If the new policy changes too much, the clip removes the extra benefit, so there is no reason to move further. **Advantage estimation:** usually done with GAE (Generalized Advantage Estimation) [8]. **Total loss:** the policy loss + a value-function loss + a small entropy bonus (to keep some exploration). **Training loop:** collect data → compute advantages → run several epochs of mini-batch updates → discard the data and repeat.
3. Advantages	Stable and reliable compared to many other policy gradient methods. Simpler to implement and tune than older methods like TRPO, while giving similar performance. Works with both discrete and continuous actions. Performs well on a wide range of tasks. Reuses each batch of data for several updates, which is more efficient than basic policy gradient. Very popular and well supported in libraries (for example, Stable-Baselines3).
4. Limitations	Still on-policy, so it needs more samples than off-policy methods like DQN or SAC. Performance depends on hyperparameters and small implementation details (clip value, learning rate, batch size). Clipping only approximately limits the change in the policy; it is not a strict guarantee. Can need a lot of compute for large problems.

Point	Details
5. Suitable applications	Robotics and continuous control, video games, simulation environments (Gymnasium tasks such as CartPole and LunarLander), autonomous systems, and fine-tuning large language models with human feedback (RLHF) [9].
6. Single-agent or multi-agent	Used widely for single-agent problems. It also has a multi-agent version, MAPPO (Multi-Agent PPO) [10], which uses a shared or centralized critic and has shown strong results in cooperative multi-agent tasks.
7. Use in AI Agents / Agentic AI	PPO is one of the best-known methods behind RLHF (Reinforcement Learning from Human Feedback). In this process, a reward model is first trained on human preferences, and then PPO fine-tunes the language model to give responses that get higher reward, while a penalty keeps it from drifting too far from the original model. In Agentic AI, similar reward-based training can be used to improve how an agent plans, selects tools or completes multi-step tasks. Newer methods related to PPO are also being used for this purpose.

Comparison table

Feature	Policy Gradient (REINFORCE)	Actor-Critic (A2C/A3C)	PPO
Type	Policy-based	Hybrid (policy + value)	Hybrid, with clipped updates
Learns	Policy only	Policy + value function	Policy + value function
Update timing	End of episode	Every step (or small batch)	After a batch, several epochs
Variance	High	Lower	Low
Stability	Low	Medium	High
Policy type	On-policy	On-policy	On-policy
Action space	Discrete or continuous	Discrete or continuous	Discrete or continuous
Main weakness	Noisy, slow learning	Can be unstable, two networks	Needs many samples, sensitive to tuning
Multi-agent version	Limited	MADDPG-style, centralized critic	MAPPO

4. Practical Implementation

4.1 Selection of Algorithm

Selected algorithm: Q-Learning (implemented from scratch using NumPy).

Reasons:

- It is simple, and every line of code can be explained.
- The Q-table makes the learning process easy to see and inspect.
- It works well for small, discrete environments.
- It directly applies the Q-update formula studied in the theory part.

4.2 Selection of Environment

Selected environment: Taxi-v3 (Gymnasium).

Note: OpenAI Gym is no longer actively maintained. Its successor is Gymnasium, so Gymnasium is used in this project [11].

Taxi-v3 is a grid world with 25 cells (5×5), based on the Taxi problem of Dietterich [12]. Its details are given in the Gymnasium documentation [13]. The taxi must go to the passenger, pick them up, drive to the destination, and drop them off. There are four special locations: Red, Green, Yellow and Blue.

Reasons for choosing it:

- It has a small, discrete state space, which suits a Q-table.
- The reward rules are clear and easy to explain.
- It has a clear goal, so success is easy to measure.
- The task needs more than one step (pickup, then drive, then drop-off), so it is better than a very trivial environment.
- It trains in a short time, even on a normal laptop.

4.3 Definition of States

A state describes the full situation of the environment at one moment. In Taxi-v3, a state is made of three things:

Component	Possible values	Count
Taxi position	Any cell of the 5×5 grid (row, column)	25
Passenger location	Red, Green, Yellow, Blue, or inside the taxi	5
Destination	Red, Green, Yellow, Blue	4

Total number of states = 25 × 5 × 4 = **500**. Gymnasium represents each state as a single integer from 0 to 499.

Gymnasium encodes the state like this [13]:

```
state = ((taxi_row × 5 + taxi_col) × 5 + passenger_location) × 4 +
destination
```

So the Q-table has size **500 × 6** (500 states × 6 actions).

4.4 Definition of Actions

The agent can choose one of **6 discrete actions**:

Action number	Meaning
0	Move South (down)
1	Move North (up)
2	Move East (right)
3	Move West (left)
4	Pick up the passenger
5	Drop off the passenger

Moving into a wall does not move the taxi (it stays in the same cell and still costs the step penalty).

4.5 Reward Mechanism

Event	Reward
Every normal step	−1
Successful drop-off at the correct destination	+20
Illegal pickup or drop-off	−10

How this shapes learning:

- The −1 per step is a time penalty. It pushes the agent to finish the task quickly instead of wandering.
- The +20 is the main goal reward.
- The −10 penalty teaches the agent not to pick up or drop off at the wrong place.

An episode ends in two ways: **terminated** (the passenger is dropped off successfully) or **truncated** (the time limit of 200 steps is reached). A perfect agent usually gets a total reward of about 7 to 9 per episode, because it needs roughly 11 to 13 steps and earns +20 at the end.

4.6 Agent Training

Training Method

The agent was trained using tabular Q-Learning, written from scratch with NumPy. The Q-table has 500 rows (states) and 6 columns (actions), and all values start at zero. In each episode, the agent follows these steps:

1. Reset the environment and get the starting state.
2. Choose an action using the ε-greedy strategy: with probability ε take a random action (explore), otherwise take the action with the highest Q-value (exploit).
3. Perform the action and receive the reward and the next state.
4. Update the Q-table using the Q-Learning rule:
   $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \cdot \max Q(s',a') - Q(s,a)\,]$
5. Move to the next state and repeat until the episode ends (successful drop-off or the 200-step limit).
6. After each episode, reduce ε a little (ε-decay).

When an episode ends with a successful drop-off, no next-state value is added, because there is no future after the goal.

4.7 Hyperparameters

Hyperparameter	Value	Meaning
Learning rate (α)	0.1	How much the old Q-value moves towards the new target
Discount factor (γ)	0.99	How much future rewards matter
Initial epsilon (ε)	1.0	Fully random at the start
Minimum epsilon	0.01	At least 1% exploration at all times
Epsilon decay	0.999 per episode	ε is multiplied by 0.999 after each episode
Training episodes	10,000	Number of training episodes
Random seed	0	For repeatable results

With a decay of 0.999, ε falls to its minimum of 0.01 after about 4,600 episodes. At around episode 2,100 (when the reward became stable, see Section 4.10), ε was still about 0.12. This means the agent learned a good policy while it was still exploring quite a lot.

Training took about 8 seconds on Google Colab. At the end of training, the average reward over the last 100 training episodes was **7.97**.

4.8 Performance Evaluation

Metrics Used

Metric	Why it is used
Total reward per episode	Shows how well the agent performs in each episode
Moving average reward (window = 100)	Smooths the noise so the learning trend is visible
Success rate (%)	Percentage of episodes where the passenger was dropped at the correct place
Average steps per episode	Shows how efficient the agent is (fewer steps is better)

4.9 Training Graphs

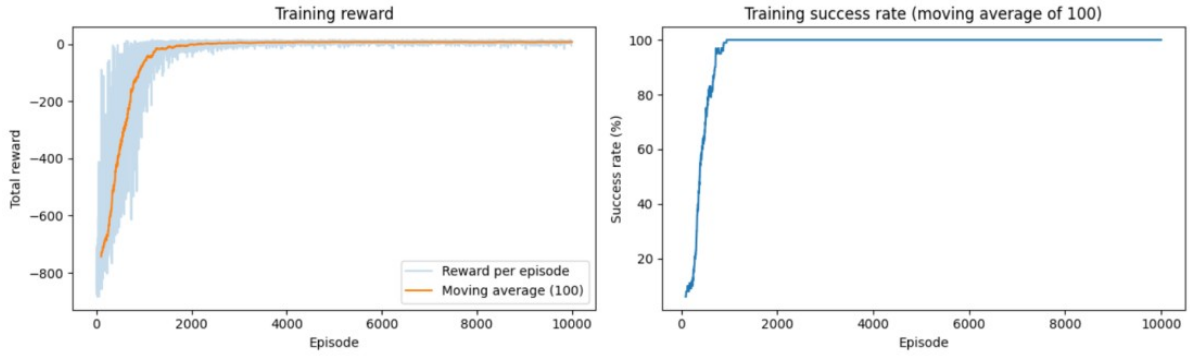


Figure 2: Training reward (left) and training success rate (right) over 10,000 episodes.

How to read the graphs:

- **Training reward (left):** In the first episodes, the total reward per episode was very low, between about −800 and −900, and the moving average started near −750. After that, it rose quickly and became stable near zero and above. The light blue line (single episodes) is noisy at the start and becomes very steady later.
- **Success rate (right):** The success rate started very low (around 5 to 10%), climbed steeply, and reached about 100% at around episode 1,000. It stayed at 100% until the end of training.

4.10 Convergence

Measure	Value
Success rate first reached about 100%	around episode 1,000 (read from the graph)
Moving average reward first reached 0	around episode 2,107 (from the code output)
Average reward of last 100 training episodes	7.97

The success rate reached 100% earlier than the reward reached zero. This is because in the early part of training the agent could already finish the task, but it still took many steps and made some illegal moves, so the reward was still negative. Over the next episodes, it learned shorter and cleaner routes, and the reward kept rising.

4.11 Testing of the Trained Agent

Test Setup

- The trained Q-table was used with **ε = 0** (pure exploitation): the agent always takes the action with the highest Q-value.
- No learning happens during testing, so the Q-table does not change.
- 100 test episodes were run.
- The test episodes use different random starting situations (different seeds) from those used in training, to keep the test fair.
- A **random agent** was also tested for 100 episodes as a baseline.

Test Results

Agent	Average reward	Average steps	Success rate
Trained Q-Learning agent (ε = 0)	**7.63**	**13.37**	**100%**
Random agent (baseline)	−758.03	194.30	7%

Observations from Testing

- The trained agent completed all 100 episodes successfully (100%), while the random agent succeeded in only 7%.
- The random agent used about 194 steps on average, which is close to the 200-step limit. Most of its episodes ended because time ran out, not because it finished the task.
- The trained agent needed only about 13 steps on average.
- The average reward of the trained agent (7.63) is exactly 21 minus the average steps (13.37). For a successful episode, the reward is +20 for the drop-off and −1 for each earlier step, so this shows the agent did not make any illegal pickup or drop-off.
- The test reward (7.63) is a little lower than the last-100 training average (7.97). This is a normal difference, because the two groups of episodes started from different random situations.

4.12 Results and Observations

Hyperparameter Experiments

To understand how the hyperparameters affect learning, five more agents were trained for 10,000 episodes each (same seed) and each was tested for 100 episodes with ε = 0.

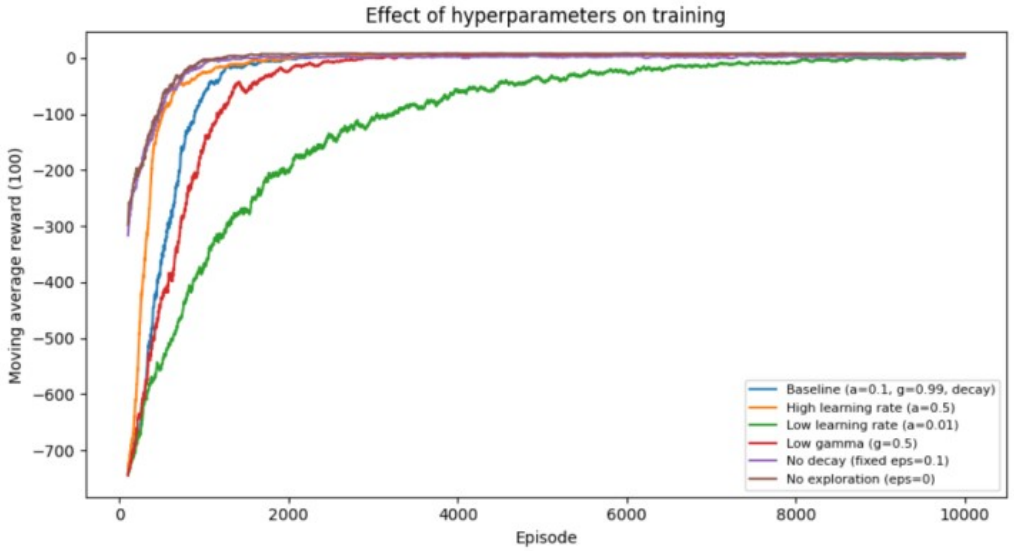


Figure 3: Effect of different hyperparameters on the moving average training reward.

Configuration	Test avg reward	Test avg steps	Test success %
Baseline (α = 0.1, γ = 0.99, ε-decay)	7.63	13.37	100
High learning rate (α = 0.5)	7.57	13.43	100
Low learning rate (α = 0.01)	−129.52	120.43	42
Low gamma (γ = 0.5)	−23.14	40.99	85
No decay (fixed ε = 0.1)	7.63	13.37	100
No exploration (ε = 0)	7.63	13.37	100

Observations

1. Learning rate (α).

- With α = 0.5, the agent learned faster than the baseline (the orange curve rises more steeply than the blue one) and reached the same test quality (100% success, 13.43 steps; the tiny difference from the baseline is not important).
- With α = 0.01, learning was very slow. The green curve was still climbing at the end of training, and the test result was poor: only 42% success and about 120 steps. The Q-values simply did not have enough time to become accurate in 10,000 episodes.
- High α worked well here because Taxi-v3 is a deterministic environment (the same action in the same state always gives the same result). In noisy environments, a very high α can make learning unstable.

2. Discount factor (γ).

- With γ = 0.5, the agent performed worse (85% success, about 41 steps, average reward −23.14). The main reward (+20) comes only at the end of the task. With a low γ, this future reward

becomes very small when it travels back through many steps, so the agent has a weak signal to follow in states far from the goal.

- With γ = 0.99, the agent values long-term reward and finds the short route.

3. Epsilon decay and exploration.

- The baseline with ε-decay, the fixed ε = 0.1, and ε = 0 all gave the same test result (7.63 reward, 13.37 steps, 100% success). This means all three learned the same good policy.
- In the training graph, the fixed ε = 0.1 and ε = 0 curves rise faster at the start than the baseline. A likely reason is that these agents take fewer random actions, so they collect fewer −10 penalties early on. The baseline starts with ε = 1.0 (fully random), so its early reward is much lower, but it explores more.
- The result for ε = 0 was a surprise, because with no exploration an agent usually gets stuck. A likely reason is that the Q-table starts at zero while all normal rewards are negative (−1 per step). So an action that has not been tried yet (value 0) looks better than an action that was tried and got a negative value. This pushes the greedy agent to try new actions on its own, which acts like built-in exploration. Taxi-v3 is also small and deterministic, which makes this easier.
- This result should not be taken as proof that exploration is unnecessary. In larger or noisy environments, or with different starting values, ε = 0 would likely fail. It is also based on one run (one seed), so repeating with other seeds would make the conclusion stronger.

4.13 What the Agent Learned

At the beginning, the agent knew nothing and behaved like a random agent, with very low rewards and many −10 penalties. Through trial and error, the Q-table slowly filled with useful values. The agent learned to go to the passenger, pick them up only at the correct place, drive to the destination, and drop them off, using about 13 steps on average. The graphs show that this learning was clear and stable.

4.14 Limitations of This Implementation

- Tabular Q-Learning works here only because Taxi-v3 has just 500 states. For large or continuous problems (such as images), a Q-table is not practical, and a method like DQN would be needed.
- Results come from a single training run for each setting, so small differences (such as 7.57 vs 7.63) should not be treated as important.

4.15 Conclusion of the Implementation

The Q-Learning agent successfully solved Taxi-v3. After 10,000 training episodes, it reached a 100% success rate with about 13 steps per episode, compared with 7% success and about 194 steps for a random agent. The experiments showed that the learning rate and the discount factor have a strong effect on learning, with α = 0.01 and γ = 0.5 giving clearly worse results than the baseline.

5. RL in AI Agents and Agentic AI

5.1 Why RL Fits Agents

An AI agent is a system that observes a situation, decides what to do, takes an action, and learns from the outcome. This is exactly the RL loop from Figure 1. Because of this, RL gives a natural way to think about, and sometimes to train, intelligent agents.

Agentic AI systems usually work on tasks with many steps, such as planning, using tools, and checking results. In such tasks, the final success often comes only at the end. RL is built for this kind of delayed reward.

5.2 Main Uses

Use	How RL applies
Learning from human feedback (RLHF)	A reward model is first trained on human preferences. Then an RL method such as PPO fine-tunes a language model to give responses that get higher reward, while a penalty stops it from drifting too far from the original model [9].
Tool and action selection	Choosing which tool to use can be treated as an RL problem. The state is the current task situation, the action is the tool or next step, and the reward is success or failure.
Planner and evaluator roles	The Actor-Critic idea (one part acts, another part judges) is similar to agent designs with a planner that proposes steps and an evaluator that checks them.
Multi-step tasks	RL helps an agent learn which early steps lead to a good final result, even when the reward comes late.
Reward-based feedback	Signals like task success, test results, or user ratings can serve as rewards to improve the agent's behaviour over time.
Multi-agent systems	When several agents work together or compete, multi-agent RL methods (such as MAPPO [10]) and ideas from game theory are used.

5.3 Challenges

- **Reward design is hard.** If the reward is badly designed, the agent may find shortcuts that earn reward without doing the real task.
- **Safety.** Exploration can be risky if the agent's actions have real-world effects, so exploration usually happens in simulation or with human approval.
- **Data and cost.** Many RL methods need a large number of interactions, which can be expensive for real systems.

5.4 Link to Our Project

Our Taxi-v3 agent is a small version of this idea. It has a clear state, a limited set of actions, and a reward signal, and it learns a policy by trial and error. Larger agentic systems use the same basic loop, but with much more complex states and actions, which is why neural-network-based methods like PPO are used.

6. Conclusion

This report studied Reinforcement Learning (RL) from basic ideas to a working implementation. It explained the main concepts, namely the agent, environment, states, actions, rewards, policy, value function, Q-value, exploration vs exploitation, episodes and the Markov Decision Process. It also compared six major algorithms: Q-Learning, SARSA, DQN, Policy Gradient methods, Actor-Critic methods and PPO.

Summary of the algorithms. The methods build on each other. Q-Learning and SARSA store values in a table and work well for small problems. DQN replaces the table with a neural network, so it can handle large state spaces. Policy Gradient methods learn the policy directly and can handle continuous actions, but their learning is noisy. Actor-Critic methods reduce this noise by adding a critic, and PPO makes training more stable by limiting how much the policy can change in one update.

Practical results. A Q-Learning agent was built from scratch and trained on Taxi-v3 for 10,000 episodes. In testing with $\varepsilon = 0$ over 100 episodes, the trained agent reached a 100% success rate, using about 13 steps and earning an average reward of 7.63. A random agent, used as a baseline, reached only 7% success, using about 194 steps with an average reward of −758.03. This shows clearly that the agent learned a useful policy through trial and error.

Hyperparameter findings. The experiments showed that the settings matter. A very low learning rate ($\alpha = 0.01$) learned too slowly, and the agent reached only 42% success in testing. A low discount factor ($\gamma = 0.5$) gave 85% success, because the agent paid too little attention to the reward at the end of the task. Changing ε-decay did not change the final result in this small, deterministic environment, but this result comes from one run and should not be taken as a general rule.

Link to AI Agents. The agent-environment loop used in this project is the same basic loop used in larger AI agents. Ideas from RL, such as reward-based feedback, learning from human preferences (RLHF with PPO) and the actor-critic structure, are connected to how modern agentic AI systems are trained and improved.

Limitations and future work. Tabular Q-Learning works only because Taxi-v3 is small (500 states). For larger or continuous problems, methods such as DQN or PPO are needed. Results are also based on a single run for each setting. In the future, the work could be extended by using several random seeds, trying SARSA and DQN on the same problems, testing PPO on a continuous-control task such as CartPole, and exploring multi-agent settings.

7. References

[1] Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

[2] Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. Machine Learning, 8(3–4), 279–292. https://link.springer.com/article/10.1007/BF00992698

[3] Rummery, G. A., & Niranjan, M. (1994). On-line Q-learning using connectionist systems (Technical Report CUED/F-INFENG/TR 166). Cambridge University Engineering Department.

[4] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533. https://www.nature.com/articles/nature14236

[5] Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8(3–4), 229–256. https://link.springer.com/article/10.1007/BF00992696

[6] Mnih, V., Badia, A. P., Mirza, M., et al. (2016). Asynchronous methods for deep reinforcement learning. Proceedings of the 33rd International Conference on Machine Learning (ICML). https://proceedings.mlr.press/v48/mniha16.pdf

[7] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347. https://arxiv.org/abs/1707.06347

[8] Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. International Conference on Learning Representations (ICLR). arXiv:1506.02438. https://arxiv.org/abs/1506.02438

[9] Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems (NeurIPS) 35. https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html

[10] Yu, C., Velu, A., Vinitsky, E., et al. (2022). The surprising effectiveness of PPO in cooperative multi-agent games. Advances in Neural Information Processing Systems (NeurIPS) 35, Datasets and Benchmarks Track. https://proceedings.neurips.cc/paper_files/paper/2022/hash/9c1535a02f0ce079433344e14d910597-Abstract-Datasets_and_Benchmarks.html

[11] Towers, M., Kwiatkowski, A., Terry, J., et al. (2024). Gymnasium: A standard interface for reinforcement learning environments. arXiv:2407.17032. https://arxiv.org/abs/2407.17032

[12] Dietterich, T. G. (2000). Hierarchical reinforcement learning with the MAXQ value function decomposition. Journal of Artificial Intelligence Research, 13, 227–303. https://www.jair.org/index.php/jair/article/view/10266

[13] Farama Foundation. Gymnasium documentation: Taxi (Toy Text environments). https://gymnasium.farama.org/environments/toy_text/taxi/

Appendix A: Code

This appendix gives the main Q-Learning code (training and testing) used in Section 4. It was run in Google Colab using Python, NumPy, Matplotlib and Gymnasium. The graphs and the hyperparameter experiments were produced by calling the same train() and evaluate() functions with different settings.

A.1 Training and Evaluation Functions

```
import numpy as np
import matplotlib.pyplot as plt
import gymnasium as gym



def train(alpha=0.1, gamma=0.99, eps_start=1.0, eps_min=0.01,
          eps_decay=0.999, episodes=10000, seed=0):
    """Train a Q-Learning agent on Taxi-v3 and return the Q-table and logs."""
    env = gym.make("Taxi-v3")
    n_states = env.observation_space.n      # 500
    n_actions = env.action_space.n          # 6
    rng = np.random.default_rng(seed)

    Q = np.zeros((n_states, n_actions))     # Q-table starts with all zeros
    epsilon = eps_start

    episode_rewards = []    # total reward of each episode
    episode_success = []    # 1 if passenger was dropped correctly, else 0

    for ep in range(episodes):
        state, _ = env.reset(seed=seed if ep == 0 else None)
        total_reward = 0
        done = False

        while not done:
            # epsilon-greedy action selection
            if rng.random() < epsilon:
                action = int(rng.integers(n_actions))   # explore
            else:
                action = int(np.argmax(Q[state]))       # exploit

            next_state, reward, terminated, truncated, _ = env.step(action)

            # Q-Learning update:
            # Q(s,a) <- Q(s,a) + alpha * [ r + gamma * max Q(s',a') - Q(s,a) ]
            # If the episode really ended (terminated), there is no next value.
            best_next = np.max(Q[next_state])
            target = reward + gamma * best_next * (1 - int(terminated))
            Q[state, action] += alpha * (target - Q[state, action])

            state = next_state
            total_reward += reward
            done = terminated or truncated

        episode_rewards.append(total_reward)
        episode_success.append(int(terminated))

        # reduce epsilon after every episode
        epsilon = max(eps_min, epsilon * eps_decay)

    env.close()
    return Q, np.array(episode_rewards), np.array(episode_success)



def evaluate(Q=None, episodes=100, seed=1000):
    """Run episodes with no exploration (epsilon = 0).
    If Q is None, a random agent is used (baseline)."""
    env = gym.make("Taxi-v3")
```

```
    rng = np.random.default_rng(seed)
    rewards, steps, successes = [], [], []

    for ep in range(episodes):
        state, _ = env.reset(seed=seed + ep)
        total_reward, n_steps, done = 0, 0, False

        while not done:
            if Q is None:
                action = int(rng.integers(env.action_space.n))
            else:
                action = int(np.argmax(Q[state]))      # pure exploitation

            state, reward, terminated, truncated, _ = env.step(action)
            total_reward += reward
            n_steps += 1
            done = terminated or truncated

        rewards.append(total_reward)
        steps.append(n_steps)
        successes.append(int(terminated))

    env.close()
    return {
        "avg_reward": float(np.mean(rewards)),
        "avg_steps": float(np.mean(steps)),
        "success_rate": 100 * float(np.mean(successes)),
    }
```

A.2 Running the Training and Testing

```
# Hyperparameters
ALPHA = 0.1          # learning rate
GAMMA = 0.99         # discount factor
EPS_START = 1.0      # start with full exploration
EPS_MIN = 0.01       # never go below 1% exploration
EPS_DECAY = 0.999    # epsilon is multiplied by this after each episode
EPISODES = 10000

Q, rewards, successes = train(ALPHA, GAMMA, EPS_START, EPS_MIN,
                              EPS_DECAY, EPISODES, seed=0)

trained = evaluate(Q, episodes=100)          # trained agent, epsilon = 0
random_agent = evaluate(None, episodes=100)  # random baseline
```

A.3 Short Explanation of the Code

Code	What it does
`Q = np.zeros((n_states, n_actions))`	Creates the Q-table (500 × 6) with all values zero, because the agent knows nothing at the start.
`if rng.random() < epsilon`	Chooses a random action with probability ε (exploration).
`np.argmax(Q[state])`	Chooses the action with the highest Q-value in the current state (exploitation).
`env.step(action)`	Performs the action and returns the next state, the reward, and whether the episode ended.
`target = reward + gamma * best_next * (1 - int(terminated))`	Calculates the target value: the reward plus the discounted best future value. If the episode has ended, there is no future value.
`Q[state, action] += alpha * (target - Q[state, action])`	Moves the old Q-value a small step (size α) towards the target. This is the Q-Learning update rule.
`epsilon = max(eps_min, epsilon * eps_decay)`	Reduces ε after each episode, but never below 0.01.
`episode_success.append(int(terminated))`	Counts an episode as successful only if the passenger was dropped at the correct place.
`evaluate(...)`	Tests the agent with ε = 0 (always the best action) and returns the average reward, average steps and success rate.
`evaluate(None, ...)`	Uses random actions, to give a baseline for comparison.

Thank You!